Setup, Generalization & Data Splits

Regression: $y \in \mathbb{R}$; classification: $y \in \{1..c\}$. Learn f s.t. $\hat{y} = f(\mathbf{x}) \approx y$ on *unseen* data (same distribution). **Overfit**: train err low, val/test high (high variance). **Underfit**: train err high (high bias). **Train** = fit params. **Val** = pick hyperparams. **Test** = once, at the end. Tuning on test ⇒ optimistic (overfit to test). ***k*-fold CV**: k models fitted; CV score = *mean* val perf over folds; then retrain on all, test once. **Leakage**: highly correlated samples (e.g. subsampled time series) split across train/val ⇒ val acc inflated. *Split first, then subsample/augment.* Estimator: bias $= \mathbb{E}[\hat{z}] - z$, var $= \mathbb{E}[(\hat{z} - \mathbb{E}\hat{z})^2]$. **Params** (learned by GD): $\mathbf{W}, \mathbf{b}$, BN γ, β, PReLU α. **Hyperparams**: LR, batch size, k, λ, dropout p, loss fn, #layers, β_1, β_2.

Diagnosis (very common short answer)

- Overfit fix: more data, data aug., *reduce* complexity, L1/L2, dropout, early stop, BN, ensembles, label smoothing. NOT more layers.
- Underfit fix: bigger model, less reg., train longer. Adding reg./dropout/less data does *not* help underfit.
- Loss oscillates/diverges: lower LR or adaptive (Adam/RMSProp). Loss flat from start w/ tiny LR (10^{-6}): raise LR.
- Sudden step drop in loss curve: LR decay (annealing).
- Smooth monotone curve = full-batch GD; noisy = minibatch/SGD (noisy grad est.).
- Reg. never aims to lower *train* loss; too much ⇒ underfit (can hurt test too).
- Optimizing accuracy directly: not differentiable (zero grad a.e.) ⇒ use CE proxy. Lower CE can coexist w/ a misclassification.

k-NN & Linear Classifiers

***k*-NN**: majority vote of k nearest (metric = hyperparam). Train = store. k=1 & pt is own neighbor ⇒ train err 0. ↑k: smoother boundary, robust to noise/outliers, *less* overfit. Pick k by CV (never by train err). Cons: memory, slow test $O(N)$, curse of dim. (L_2 in pixel space ≠ similarity). Low bias / high variance; linear clf: high bias / low variance. **Linear**: $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$, $\mathbf{W} \in \mathbb{R}^{c \times n}$, rows = class templates; predict $\arg\max_j z_j$. 2-class boundary $(\mathbf{w}_1 - \mathbf{w}_2)^T\mathbf{x} = 0$. **Hinge/SVM**: $\sum_{j \neq y} \max(0, 1 + a_j - a_y)$; margin 1 WLOG (rescale $\mathbf{W}, \mathbf{b}$).

Least Squares & Matrix Calculus

$$\mathcal{L} = \tfrac{1}{2}\|\mathbf{y} - \mathbf{X}\theta\|^2 = \tfrac{1}{2}(\mathbf{y}^T\mathbf{y} - 2\mathbf{y}^T\mathbf{X}\theta + \theta^T\mathbf{X}^T\mathbf{X}\theta)$$

$$\nabla_\theta \mathcal{L} = \mathbf{X}^T\mathbf{X}\theta - \mathbf{X}^T\mathbf{y} \;\Rightarrow\; \theta^* = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

GD: $\theta \leftarrow \theta - \epsilon(\mathbf{X}^T\mathbf{X}\theta - \mathbf{X}^T\mathbf{y})$. Ridge ($+\frac{\lambda}{2}\|\theta\|^2$): $\theta \leftarrow (1 - \epsilon\lambda)\theta - \epsilon\nabla\mathcal{L}$; closed form $(\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^T\mathbf{y}$.

Denominator layout: $\nabla_{\mathbf{x}} y$ same shape as $\mathbf{x}$; $\nabla_{\mathbf{x}}\mathbf{y} \in \mathbb{R}^{m \times n} = \mathbf{J}^T$.

- $\nabla_{\mathbf{x}}(\mathbf{a}^T\mathbf{x}) = \mathbf{a}$; $\nabla_{\mathbf{x}}(\mathbf{x}^T\mathbf{A}\mathbf{x}) = (\mathbf{A} + \mathbf{A}^T)\mathbf{x}$; $\nabla_{\mathbf{x}}(\mathbf{W}\mathbf{x}) = \mathbf{W}^T$
- $\nabla_{\mathbf{A}}(\mathbf{z}^T\mathbf{A}\mathbf{x}) = \mathbf{z}\mathbf{x}^T$; $\nabla_{\mathbf{W}}\|\mathbf{W}\|_F^2 = 2\mathbf{W}$; $\nabla_{\mathbf{X}} \log|\mathbf{X}| = \mathbf{X}^{-T}$
- $\nabla_{\mathbf{x}}\|\mathbf{x}\|_2 = \mathbf{x}/\|\mathbf{x}\|_2$; quotient: $(f/g)' = (f'g - fg')/g^2$

Chain rule: $\mathbf{x} \to \mathbf{y} \to \mathbf{z}$: $\nabla_{\mathbf{x}}\mathbf{z} = \nabla_{\mathbf{x}}\mathbf{y}\,\nabla_{\mathbf{y}}\mathbf{z}$ **(right-to-left; check dims)**. Hessian $H_{ij} = \partial^2 f/\partial x_i \partial x_j$. Geometric: $\sum_{j=0}^{n-1} r^j = \frac{1-r^n}{1-r}$.

MLE & Probabilistic Losses

$\theta^* = \arg\max \prod_i p(y^{(i)}|\mathbf{x}^{(i)}, \theta) = \arg\min -\sum_i \log p(\cdot)$ ($p(\mathbf{x})$ indep. of θ; log monotone). **Bernoulli/coin**: $\theta^* = \frac{n_H}{n_H + n_T}$.

Poisson $\frac{\lambda^k e^{-\lambda}}{k!}$: $\ell = -n\lambda + \ln\lambda \sum x_j - \sum \ln x_j!$, $\hat{\lambda} = \bar{x}$. **Gaussian** noise ⇒ MSE. **BCE = MLE**: $p(y|\mathbf{x}) = \hat{y}^y(1-\hat{y})^{1-y}$ ⇒ $-\log L = -\sum[y \log\hat{y} + (1-y)\log(1-\hat{y})]$.

LDA→softmax: $p(\mathbf{x}|c) = \mathcal{N}(\mu_c, \Sigma)$, prior π_c: $p(c|\mathbf{x}) = \text{softmax}$ with $\mathbf{w}_c = \Sigma^{-1}\mu_c$, $b_c = -\frac{1}{2}\mu_c^T\Sigma^{-1}\mu_c + \log\pi_c$ (the $\mathbf{x}^T\Sigma^{-1}\mathbf{x}$ term cancels).

Softmax Classifier

$a_j = \mathbf{w}_j^T\mathbf{x} + b_j$, $p_j = \frac{e^{a_j}}{\sum_k e^{a_k}}$.
$\mathcal{L}_i = \log\sum_j e^{a_j} - a_{y_i} = -\log p_{y_i}$.

$$\frac{\partial \mathcal{L}_i}{\partial a_k} = p_k - \mathbb{I}[k{=}y_i] \quad \text{(vector: } \mathbf{p} - \mathbf{e}_{y_i}\text{)}$$

$\frac{\partial \mathcal{L}}{\partial \mathbf{w}_k} = \sum_i (p_k^{(i)} - \mathbb{I}[k{=}y_i])\mathbf{x}^{(i)}$,
$\frac{\partial \mathcal{L}}{\partial b_k} = \sum_i (p_k^{(i)} - \mathbb{I}[k{=}y_i])$; matrix: $\nabla_{\mathbf{W}} = (\mathbf{p} - \mathbf{e}_y)\mathbf{x}^T$. **Exam:** loss at init (small rand. $\mathbf{W}$) $\approx \ln C$ (C=10: 2.30; C=7: 1.95). CE of one example $= -\log\hat{y}_{\text{true}}$ (e.g. $-\log 0.3$). Overflow: subtract $\max_j a_j$ from all scores. Label smoothing: targets $1-\epsilon$ and $\frac{\epsilon}{k-1}$ (regularizer; robust to label errors).

Neural Nets

$\mathbf{h}_1 = f(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1), \ldots, \mathbf{z} = \mathbf{W}_N\mathbf{h}_{N-1} + \mathbf{b}_N$ (no act. on output). **Params**: $\sum(n_{in}n_{out} + n_{out})$; e.g. 3→4→2: 26. Residual $\mathbf{h}_1 = f(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{W}_s\mathbf{x}$: $2mn + m + ml + l$. Linear f collapses to one linear map ($\mathbf{W} = \mathbf{W}_N \cdots \mathbf{W}_1$, $\mathbf{b} = \mathbf{b}_N + \mathbf{W}_N\mathbf{b}_{N-1} + \ldots$); only useful for low-rank (autoencoder ≈ SVD). Nonlinear hidden layers learn features that make data linearly separable. Universal approx: 1 hidden layer, enough units. **XOR**: linear LS fails; $g(\mathbf{x}) = \mathbf{w}^T\max(0, \mathbf{W}^T\mathbf{x} + \mathbf{c})$, $\mathbf{W} = \begin{bmatrix}1 & 1\\ 1 & 1\end{bmatrix}$, $\mathbf{c} = [0, -1]^T$, $\mathbf{w} = [1, -2]^T$ works.

Activation Functions

	$f'(x)$	Sat.	ZZ	Diff	Id_0
$\sigma = \frac{1}{1+e^{-x}}$	$\sigma(1-\sigma) \le \frac{1}{4}$	$\pm\infty$	Y	Y	N
$\tanh = 2\sigma(2x) - 1$	$1 - \tanh^2$	$\pm\infty$	N	Y	Y
ReLU $\max(0, x)$	$\mathbb{I}(x > 0)$	$x < 0$	Y	N	N
Leaky/PReLU $\max(\alpha x, x)$	1 / α	no	N	N	N
ELU x / $\alpha(e^x - 1)$	1 / αe^x	$-\infty$	N	N*	N
Softplus $\log(1+e^x)$	$\sigma(x) = \frac{e^x}{1+e^x}$	$-\infty$	Y	Y	N
e^x	e^x	$-\infty$	Y	Y	N
GELU $x\Phi(x)$	$\Phi + x\phi$ ($\frac{1}{2}$ at 0)	$-\infty$	N	Y	N

ZZ = zig-zag (outputs all $\ge 0 \Rightarrow \partial f/\partial \mathbf{w} = f' \cdot \mathbf{h}$ entries share sign). Id_0: $f(0) = 0, f'(0) = 1$. *ELU diff. iff $\alpha = 1$. PReLU: $\partial f/\partial\alpha = x\,\mathbb{I}(x < 0)$; learnable α avoids dead units & zig-zag. **Exam:** Vanishing grads: sigmoid, tanh (saturation), ReLU ($x < 0$). Step/sign fn: $f' = 0$ a.e. ⇒ no learning. Sigmoid outputs ~ 0.5 (not 0-centered). ReLU: fast, no sat. for $x > 0$, dead units. **Output↔loss**: σ+MSE: $\partial/\partial z = -(y - \sigma)\sigma(1 - \sigma) \to 0$ when very wrong. σ+CE: $\partial/\partial z = \sigma(z) - y$ (learns). Pair CE with sigmoid/softmax.

Backprop

Forward: compute & cache node values. Backward: downstream = local × upstream. **add** distributes; **mult** swaps ($\partial_x = y$·up); **max** routes to the winner. E.g. $f = (x + y)z$, $(2, 3, 4)$: $\partial_z = 5$, $\partial_x = \partial_y = 4$. Sigmoid gate: local $= \sigma(1 - \sigma)$ (e.g. $0.73 \cdot 0.27 = 0.20$). Write intermediate vars, label graph edges, give dims. **Paths merge ⇒ sum** (shared weights, siamese nets, residual, $\mathbf{x}$ used twice). **Elementwise** $\mathbf{h} = f(\mathbf{a})$: $\delta_{\mathbf{a}} = f'(\mathbf{a}) \odot \delta_{\mathbf{h}}$. **Affine** $\mathbf{c} = \mathbf{W}\mathbf{x} + \mathbf{b}$: $\delta_{\mathbf{x}} = \mathbf{W}^T\delta_{\mathbf{c}}$, $\delta_{\mathbf{W}} = \delta_{\mathbf{c}}\mathbf{x}^T$, $\delta_{\mathbf{b}} = \delta_{\mathbf{c}}$. Row-vector $\mathbf{W}_2 \in \mathbb{R}^{1 \times d}$: $\delta_{\mathbf{W}_2} = \delta_z\,\mathbf{a}_1^T$. **Template** (MLP, $\hat{y} = \sigma(z_2)$): $\delta_{z_2} = \hat{y}(1 - \hat{y})\delta_{\hat{y}}$ [$= \hat{y} - y$ for BCE]; $\delta_{\mathbf{W}_2} = \delta_{z_2}\mathbf{a}_1^T$, $\delta_{\mathbf{b}_2} = \delta_{z_2}$; $\delta_{\mathbf{a}_1} = \mathbf{W}_2^T\delta_{z_2}$; $\delta_{\mathbf{z}_1} = \mathbb{I}(\mathbf{z}_1 > 0) \odot \delta_{\mathbf{a}_1}$; $\delta_{\mathbf{W}_1} = \delta_{\mathbf{z}_1}\mathbf{x}^T$, $\delta_{\mathbf{b}_1} = \delta_{\mathbf{z}_1}$. **Siamese** ($\mathbf{h}_p = \mathbf{W}_1\mathbf{x}_p + \mathbf{b}_1$, $\mathbf{h}_q = \mathbf{W}_1\mathbf{x}_q + \mathbf{b}_1$): $\delta_{\mathbf{W}_1} = \delta_{\mathbf{h}_p}\mathbf{x}_p^T + \delta_{\mathbf{h}_q}\mathbf{x}_q^T$, $\delta_{\mathbf{b}_1} = \delta_{\mathbf{h}_p} + \delta_{\mathbf{h}_q}$. Residual $\mathbf{z} = \mathbf{W}_3 f(\mathbf{W}_1\mathbf{x}) + \mathbf{W}_2\mathbf{x}$: $\delta_{\mathbf{x}} = \mathbf{W}_1^T[f' \odot \mathbf{W}_3^T\delta_{\mathbf{z}}] + \mathbf{W}_2^T\delta_{\mathbf{z}}$. Nullspace $\mathbf{W}_1\mathbf{z} = 0$: replacing $\mathbf{x} \to \mathbf{x} + \mathbf{z}$ changes only $\delta_{\mathbf{W}_1} = \delta_{\mathbf{h}}(\mathbf{x} + \mathbf{z})^T$.

Upstream (loss) gradients

- $\frac{1}{2}\|\mathbf{z} - \mathbf{y}\|^2$: $\mathbf{z} - \mathbf{y}$. $\|\mathbf{h} - \mathbf{z}\|^2$: $2(\mathbf{h} - \mathbf{z})$
- BCE wrt $\hat{y}$: $-\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}$; weighted $-[\alpha y \log\hat{y} + \beta(1-y)\log(1-\hat{y})]$: $-\frac{\alpha y}{\hat{y}} + \frac{\beta(1-y)}{1-\hat{y}}$. Balance: weight ∝ 1/class size (10k neg : 1k pos ⇒ α=10, β=1).
- Hinge $\sum_i \max(0, 1 - y_i\hat{y}_i)$: $-y_i$ where active ($\hat{y} \in (0, 1)$, $y \in \{0, 1\}$ ⇒ always active).
- Cosine $s = \frac{\mathbf{z}^T\hat{\mathbf{z}}}{\|\mathbf{z}\|\|\hat{\mathbf{z}}\|}$: $\frac{\partial s}{\partial \mathbf{z}} = \frac{\hat{\mathbf{z}}}{\|\mathbf{z}\|\|\hat{\mathbf{z}}\|} - \frac{s\,\mathbf{z}}{\|\mathbf{z}\|^2}$. s=1 same dir., 0 orthogonal, −1 opposite.
- $(\log|\mathbf{h}\mathbf{h}^T + \mathbf{I}| - c)^2$: with $\mathbf{M} = \mathbf{h}\mathbf{h}^T + \mathbf{I}$: $\delta_{\mathbf{h}} = 4(\cdot)\mathbf{M}^{-T}\mathbf{h}$.

Penalty gradients (add to ∇ of data loss)

- $\frac{\lambda}{2}\|\mathbf{w}\|_2^2 \to \lambda\mathbf{w}$; $\lambda\|\mathbf{W}\|_F^2 \to 2\lambda\mathbf{W}$; $\lambda\|\mathbf{w}\|_1 \to \lambda\,\text{sign}(\mathbf{w})$
- $\|\mathbf{A}\|_\infty = \max_k \sum_j |A_{kj}|$ (max abs row sum): ∇ = zeros except row j^* (the max row) $= \text{sign}(\mathbf{A}_{j^*,:})$
- $\text{LSE}(\mathbf{x}) = \ln\sum_i e^{x_i} \to \text{softmax}(\mathbf{x})$
- $\|\mathbf{w}_1 - \mathbf{w}_k\|^2$ (tie layers): $\pm 2(\mathbf{w}_1 - \mathbf{w}_k)$. $\|\mathbf{h}\|_1$: sparse activations. $\|\mathbf{h} - \mathbf{1}\|^2$: acts → 1.
- Elastic net $\frac{1}{2n}\|\mathbf{y} - \mathbf{X}\theta\|^2 + \lambda[\alpha\|\theta\|_1 + \frac{1-\alpha}{2}\|\theta\|^2]$: $\nabla = \frac{1}{n}\mathbf{X}^T(\mathbf{X}\theta - \mathbf{y}) + \lambda[\alpha\,\text{sign}\,\theta + (1-\alpha)\theta]$. α=1: L1 (sparse); α=0: L2 (shrink). Combines sparsity with stability for correlated features.

Weight Initialization

All zero (no bias, tanh): zero grads, no learning. **All same const**: symmetry not broken; all units in a layer get identical grads (stay equal). **Small** (deep): acts shrink each layer → 0 ⇒ $\delta_{\mathbf{W}} = \delta\mathbf{h}^T \approx 0$. **Large**: tanh saturates (vanishing), ReLU explodes. $\text{var}(wh) = \mathbb{E}^2[w]\text{var}(h) + \mathbb{E}^2[h]\text{var}(w) + \text{var}(w)\text{var}(h)$ ⇒ $\text{var}(h_i) = n_{in}\text{var}(w)\text{var}(h_{i-1})$ (zero-mean, indep.); fwd wants $\text{var}(w) = \frac{1}{n_{in}}$, bwd $\frac{1}{n_{out}}$. **Xavier**: $\text{var}(w) = \frac{2}{n_{in}+n_{out}}$ (tanh). **He**: $\frac{2}{n_{in}}$ (ReLU kills half).

Batch / Layer Normalization

Per unit over minibatch: $\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}$, $y = \gamma\hat{x} + \beta$ (γ, β *learned*; can undo BN). Placed before nonlinearity. Test: running avg. of μ, σ^2. **Exam:** BN is *affine/linear* per unit, learnable, adds noise (batch estimates) ⇒ mild regularizer, speeds training (higher LR), robust to init (acts can't vanish/explode; reduces internal covariate shift). **Backprop**: $\partial_\beta = \sum_j \delta_{y_j}$, $\partial_\gamma = \sum_j \delta_{y_j}\hat{x}_j$, $\delta_{\hat{x}_j} = \gamma\delta_{y_j}$;
$\delta_{\sigma^2} = \sum_j \delta_{\hat{x}_j}(x_j - \mu)\cdot(-\frac{1}{2})(\sigma^2 + \varepsilon)^{-3/2}$;
$\delta_\mu = -\frac{\sum_j \delta_{\hat{x}_j}}{\sqrt{\sigma^2+\varepsilon}} - \frac{2}{m}\sum_j(x_j - \mu)\,\delta_{\sigma^2}$;
$\delta_{x_j} = \frac{\delta_{\hat{x}_j}}{\sqrt{\sigma^2+\varepsilon}} + \frac{2(x_j-\mu)}{m}\delta_{\sigma^2} + \frac{1}{m}\delta_\mu$ (3 paths).
LayerNorm: stats over the D units of *one example* (not over batch); same at train/test; better for small batches, RNNs/sequences, transformers.

Regularization

Reduce generalization (not train) error: ↑bias, ↓variance. Early stop, norm penalties, aug., dropout, ensembles, multitask, BN, noise, label smoothing are all regularizers. **L2 = weight decay**: $\mathbf{w} \leftarrow (1 - \epsilon\lambda)\mathbf{w} - \epsilon\nabla J$ (shrink by const. factor each step). Quadratic approx.: $\tilde{w}_i = \frac{h_i}{h_i + \lambda}w_i^*$ (shrinks low-curvature dirs most). **L1 ⇒ sparsity**: with $J \approx J(\mathbf{w}^*) + \frac{1}{2}(\mathbf{w} - \mathbf{w}^*)^T\mathbf{H}(\mathbf{w} - \mathbf{w}^*)$, $\mathbf{H} = \text{diag}(h_i)$:

$\nabla_{w_i} = h_i(w_i - w_i^*) + \lambda \operatorname{sign}(w_i) = 0 \Rightarrow$ $w_i = \operatorname{sign}(w_i^*)\max\left(|w_i^*| - \frac{\lambda}{h_i}, 0\right)$ (exactly 0 if $|w_i^*| \le \lambda/h_i$). Use L1 to "shrink effective model size" (L2/dropout don't zero weights). **Early stopping**: plot loss vs. iterations; train loss keeps ↓; val loss U-shaped; stop at val minimum. **Data aug.**: crops, shifts, rotations, scaling, color, blur, flips *only if label-invariant* (no flips for digits). Helps class imbalance (augment minority). **Ensembles**: models make partly independent errors ⇒ averaging wrong only when many err together. Bagging: resample w/ replacement. **Multitask**: shared trunk + heads. **Transfer**: freeze most of pretrained net, train last layers. **PEFT**: LoRA ($\mathbf{W}_0 + \mathbf{BA}$, low rank per layer), prefix tuning, small hypernetwork; backbone frozen.

Dropout

Train: keep unit w.p. p: `M=(rand<p)`. Test: all units, scale acts by p. **Inverted**: `M=(rand<p)/p` in training so $\mathbb{E}$[act] matches test; test unchanged (no extra cost). Why it works: ≈ bagging 2^N weight-sharing subnets; no co-adaptation/over-reliance on few inputs; redundant features. Dropout at val/test time: never.

SGD, Momentum, Nesterov

GD $\theta \leftarrow \theta - \epsilon\mathbf{g}$ ($-\mathbf{g}$ = steepest descent from 1st-order Taylor). Too large ϵ: bounce/diverge; too small: slow. On quadratic: $J(\theta - \epsilon\mathbf{g}) \approx J - \epsilon\mathbf{g}^T\mathbf{g} + \frac{\epsilon^2}{2}\mathbf{g}^T\mathbf{H}\mathbf{g}$ (big curvature along $\mathbf{g}$ ⇒ step can *increase* J). Minibatch $\mathbf{g} = \frac{1}{m}\sum_{i\in B}\nabla\ell_i$: unbiased, noisy (not exact grad); noise helps escape. Full batch = exact. **Momentum** ($\mathbf{v}$=0, α=0.9):

$$\boxed{\mathbf{v} \leftarrow \alpha\mathbf{v} - \epsilon\mathbf{g},\ \theta \leftarrow \theta + \mathbf{v}}$$

. Keeps extra state $\mathbf{v}$ (running avg. of grads). Averages out zig-zag; carries through flat regions; at a min $\mathbf{g} \approx 0$ but $\mathbf{v} \neq 0$ ⇒ can leave sharp local minima, prefers wide/flat minima (generalize better). **Nesterov**:

$$\boxed{\mathbf{v} \leftarrow \alpha\mathbf{v} - \epsilon\nabla J(\theta + \alpha\mathbf{v}),\ \theta \leftarrow \theta + \mathbf{v}}$$

(look ahead, then correct).
$\nabla J(\theta + \alpha\mathbf{v}) \approx \nabla J(\theta) + \alpha\mathbf{H}\mathbf{v}$: extra $-\epsilon\alpha\mathbf{H}\mathbf{v}$ opposes the update in high curvature ⇒ less overshoot. **Reparam.** $\tilde\theta = \theta + \alpha\mathbf{v}$: $\mathbf{v}_{new} = \alpha\mathbf{v}_{old} - \epsilon\nabla J(\tilde\theta_{old})$, $\tilde\theta_{new} = \tilde\theta_{old} + \mathbf{v}_{new} + \alpha(\mathbf{v}_{new} - \mathbf{v}_{old})$. *Proof*: $\tilde\theta_{new} = \theta_{new} + \alpha\mathbf{v}_{new} = \theta_{old} + \mathbf{v}_{new} + \alpha\mathbf{v}_{new} = \tilde\theta_{old} - \alpha\mathbf{v}_{old} + (1+\alpha)\mathbf{v}_{new}$. Code: `p_old=p; p=a*p-eps*g; x+=p+a*(p-p_old)`.

Unrolled forms ($\mathbf{v}_0 = 0$)

$\mathbf{v}_t = -\epsilon\sum_{i=1}^t \alpha^{t-i}\mathbf{g}_i$;
$\theta_t = \theta_0 + \sum_{j=1}^t \mathbf{v}_j = \theta_0 - \epsilon\sum_{j=1}^t\sum_{i=1}^j \alpha^{j-i}\mathbf{g}_i$
$= \theta_0 - \epsilon\sum_{i=1}^t \frac{1-\alpha^{t-i+1}}{1-\alpha}\mathbf{g}_i$. If $\mathbb{E}\mathbf{g}_i = \mu$, $\mathbb{E}\theta_0 = 0$:

$$\mathbb{E}\theta_t = -\frac{\epsilon\mu}{1-\alpha}\left[t - \frac{\alpha(1-\alpha^t)}{1-\alpha}\right]$$

If $\mathbb{E}\mathbf{g}_i\mathbf{g}_j^T = \sigma^2\mathbf{I}\,\delta_{ij}$, zero mean:
$\operatorname{Cov}(\mathbf{v}_t) = \epsilon^2\sigma^2\frac{1-\alpha^{2t}}{1-\alpha^2}\mathbf{I}$ (cross terms vanish).
Nesterov (reparam form):
$\theta_t = \theta_0 + \alpha\mathbf{v}_t + \sum_{j=1}^t \mathbf{v}_j$.

Adaptive LR (per-parameter)

Annealing: $\epsilon \leftarrow 0.1\epsilon$, *scheduled* (every N iters/epochs) or *event-based* (loss plateaus). **Adagrad**: $\mathbf{a} \leftarrow \mathbf{a} + \mathbf{g}\odot\mathbf{g}$, $\theta \leftarrow \theta - \frac{\epsilon}{\sqrt{\mathbf{a}}+\nu}\odot\mathbf{g}$ (ν=10^{-7}). Large past grads in a coord. ⇒ smaller step there; boosts flat directions. Con: $\mathbf{a}$ only grows ⇒ eff. LR monotonically ↓ (can stall early). Const. g: step = $\epsilon/\sqrt{t}$. **RMSProp**: $\mathbf{a} \leftarrow \beta\mathbf{a} + (1-\beta)\mathbf{g}\odot\mathbf{g}$ (β=0.99), same step. EMA forgets old grads ⇒ $\mathbf{a}$ can shrink or grow (fixes Adagrad). Const. g: step → $\epsilon\ \operatorname{sign}(g)$. *Unroll*: $\mathbf{a}_t = (1-\beta)\sum_{i=1}^t \beta^{t-i}\mathbf{g}_i^2$; e.g. $\mathbf{a}_1 = (1-\beta)\mathbf{g}_1^2$, $\mathbf{a}_2 = \beta(1-\beta)\mathbf{g}_1^2 + (1-\beta)\mathbf{g}_2^2$, $\theta_2 = \theta_0 - \frac{\epsilon\mathbf{g}_1}{\sqrt{\mathbf{a}_1}+\nu} - \frac{\epsilon\mathbf{g}_2}{\sqrt{\mathbf{a}_2}+\nu}$. **RMSProp+mom.**: $\mathbf{v} \leftarrow \alpha\mathbf{v} - \frac{\epsilon}{\sqrt{\mathbf{a}}+\nu}\odot\mathbf{g}$, $\theta \leftarrow \theta + \mathbf{v}$. **Adam** ($\beta_1$=0.9, β_2=0.999; momentum step + RMSProp step):

$$\mathbf{v} \leftarrow \beta_1\mathbf{v} + (1-\beta_1)\mathbf{g}, \quad \mathbf{a} \leftarrow \beta_2\mathbf{a} + (1-\beta_2)\mathbf{g}\odot\mathbf{g}$$

$$\tilde{\mathbf{v}} = \frac{\mathbf{v}}{1-\beta_1^t}, \quad \tilde{\mathbf{a}} = \frac{\mathbf{a}}{1-\beta_2^t}, \quad \theta \leftarrow \theta - \frac{\epsilon}{\sqrt{\tilde{\mathbf{a}}}+\nu}\odot\tilde{\mathbf{v}}$$

Bias corr.: $\mathbf{v}_t = (1-\beta_1)\sum_{i=1}^t \beta_1^{t-i}\mathbf{g}_i$ ⇒ $\mathbb{E}\mathbf{v}_t = (1-\beta_1)\frac{1-\beta_1^t}{1-\beta_1}\mu = (1-\beta_1^t)\mu$; same for $\mathbf{a}$ with $s = \mathbb{E}g^2$. Init at 0 biases moments low; correction enlarges early 2nd moment so first steps aren't huge. $t \to \infty$: $\beta^t \to 0$, correction → 1. First Adam step (t=1): $\tilde{\mathbf{v}} = \mathbf{g}$, $\tilde{\mathbf{a}} = \mathbf{g}^2$ ⇒ $\Delta\theta \approx -\epsilon\ \operatorname{sign}(\mathbf{g})$ per coord. (scale-free; w/o correction: $\frac{0.1}{\sqrt{0.001}} \approx 3.2\times$ bigger). Robust to hyperparams; state = $\mathbf{v}, \mathbf{a}, t$. **AdamW**: L2 inside Adam ($\mathbf{g} = \nabla J + \lambda\theta$) gets divided by $\sqrt{\mathbf{a}}$ ⇒ different eff. reg. per parameter (not intended). Decouple: $\theta \leftarrow \theta - \alpha\,\text{AdamUpdate}(\nabla J) - \alpha\lambda\theta$. (For plain SGD, L2 ≡ weight decay.) Generalizes better; use over Adam.

Optimizer Reasoning (exam favorites)

- **Contour matching** at same point, same ϵ: *GD+mom*: dominated by steep (large-grad) dir., largest step, zig-zags across narrow valley. *Adagrad*: biased toward small-grad (flat) dir., smallest step. *Adam*: in between, not dominated by either axis. *RMSProp*: smooth path along valley. *SGD*: ⊥ contours, oscillates in steep dir. longest. *Momentum* arrow = $-\epsilon\mathbf{g}$ + $\alpha\times$previous step.
- **Plateau "trap"**: mom. & Adam carry accumulated velocity through; update size mom > Adam > Adagrad (Adagrad's $\mathbf{a}$ big from slope).
- **Mom. vs. Nesterov after steep descent**: Nesterov looks ahead at $\theta + \alpha\mathbf{v}$ (past the min, grad reverses) ⇒ brakes, less overshoot, converges faster.
- **At a local min** ($\mathbf{g} = 0$): mom. step = $\alpha\mathbf{v}$. Nesterov bigger if look-ahead point is past the bump (grad pushes same way), smaller if look-ahead still in the basin (grad opposes).
- **Revisit same** θ (full batch, fixed ϵ): GD same step; Adagrad smaller; RMSProp/Adam/mom. can't determine (depends on history).
- SGD+mom. *more* likely to leave a local optimum than SGD.
- Adagrad eff. LR is non-increasing: **True**. RMSProp fixes Adagrad's vanishing LR via EMA.

Reading Descent Pictures (why)

Key idea: *what sets the step direction?*

- **GD**: step $\propto -\mathbf{g}$ ⇒ ⊥ to contour lines, length $\propto$ slope. Narrow valley: $\mathbf{g}$ points mostly *across* the valley ⇒ zig-zag, slow progress along it; crawls on plateaus/near saddles (small $\mathbf{g}$).
- **Momentum**: $\Delta\theta = \alpha\mathbf{v}_{prev} - \epsilon\mathbf{g}$ (vector sum: last step + new grad). Consistent components (along valley) add up ($\le \frac{1}{1-\alpha} = 10\times$); alternating ones (across) cancel ⇒ straighter, faster path. Signs: overshoots the min, swings wide on curves, path keeps going after grad changes; can coast into a *different* basin.
- **Nesterov**: same shape, tighter turns, less overshoot (grad taken at look-ahead pt corrects early).
- **Adagrad/RMSProp**: divide each coord. by its RMS grad ⇒ every coord. moves $\approx \epsilon$ regardless of slope ⇒ direction ≈ $\operatorname{sign}(\mathbf{g})$: *not* ⊥ contours, heads ∼ 45°, then straight along flat axis (amplifies small-grad coords., damps steep ones) ⇒ no zig-zag, escapes saddles/plateaus. Adagrad: steps *shrink* over time; RMSProp keeps speed (reaches min sooner).
- **Adam**: RMSProp-style scaling of the *momentum* ⇒ smooth straight path, sign-like direction, slight overshoot/curve from momentum.
- **Newton**: $\mathbf{H}^{-1}$ rescales elongated contours into circles ⇒ points straight at the quadratic's min.

Lecture demos (2-D loss w/ saddle, local & global min): start A: all reach global min; mom./Nesterov overshoot then curl back. Start B: GD stalls near saddle; mom. & Nesterov roll into the *same local* min; Adagrad & RMSProp reach the *global* min (RMSProp faster); Adam slightly slower than RMSProp. **Checklist**: ⊥ contours, steep-axis zig-zag → GD. Continues prior direction/overshoot → momentum. Diagonal, equal-sized axis moves, no zig-zag → adaptive. Steps keep shrinking → Adagrad.

2nd-Order Methods & GD Challenges

1st-order: use $\mathbf{g}$ only, need LR. 2nd-order: use $\mathbf{H} = \nabla^2 J$ (curvature): flat ⇒ big step, high curvature ⇒ small step. **Newton**: $J \approx J_0 + (\theta-\theta_0)^T\mathbf{g} + \frac{1}{2}(\theta-\theta_0)^T\mathbf{H}(\theta-\theta_0) \Rightarrow$

$$\boxed{\theta \leftarrow \theta - \mathbf{H}^{-1}\mathbf{g}}$$

; no LR; exact in 1 step for quadratics (1-D: $\theta - f'/f''$). Neg. eigenvalues ⇒ ascent steps; fix $(\mathbf{H} + \alpha\mathbf{I})^{-1}$ ($\alpha \to \infty$: GD w/ LR $1/\alpha$). Inverting $\mathbf{H}$: $O(N^3)$, impractical. **Quasi-Newton (BFGS)**: update $\mathbf{H}^{-1}$ recursively, $\mathbf{s} = \theta_k - \theta_{k-1}$, $\mathbf{y} = \nabla J_k - \nabla J_{k-1}$: $\mathbf{H}_k^{-1} = (\mathbf{I} - \frac{\mathbf{s}\mathbf{y}^T}{\mathbf{y}^T\mathbf{s}})\mathbf{H}_{k-1}^{-1}(\mathbf{I} - \frac{\mathbf{y}\mathbf{s}^T}{\mathbf{y}^T\mathbf{s}}) + \frac{\mathbf{s}\mathbf{s}^T}{\mathbf{y}^T\mathbf{s}}$. **L-BFGS**: only last m pairs. Needs full/large batches. **CG**: directions $\mathbf{g}_k^T\mathbf{H}\mathbf{g}_{k-1} = 0$; "Hessian-free". **Exploding grads**: cliffs (repeated multiplication, RNNs) ⇒ *gradient clipping* (cap $\|\mathbf{g}\|$). **Vanishing**: $\mathbf{W}^t = \mathbf{U}\Lambda^t\mathbf{U}^{-1}$, grad along $\mathbf{u}_i$ scaled by λ_i^t ($|\lambda| < 1$ vanish, > 1 explode); fix via architecture, init, normalization.

Worked Example: Steep vs. Flat Axis

Constant $\mathbf{g} = (4, 0.1)$ (steep θ_1, flat θ_2), $\epsilon = 0.1$, $\alpha = 0.9$, $\nu \approx 0$. Steps $\Delta\theta$:

- **GD**: $-(0.4, 0.01)$ every step; ratio 40:1, almost purely steep axis.
- **Momentum**: $-(0.4, 0.01)$, then $-(0.76, 0.019)$, $\ldots \to \frac{\epsilon\mathbf{g}}{1-\alpha} = -(4, 0.1)$. Grows 10×, same direction as GD.
- **Adagrad**: $\mathbf{a}_1 = (16, 0.01)$: $-(0.1, 0.1)$ (45°!). $\mathbf{a}_2 = (32, 0.02)$: $-(0.071, 0.071)$; step t: $\frac{\epsilon}{\sqrt{t}}$.
- **RMSProp** (β=0.99): $\mathbf{a}_1 = 0.01\mathbf{g}^2$ ⇒ $-(1, 1)$ (huge first step: no bias correction); $\mathbf{a}_2 \approx 0.0199\mathbf{g}^2$: $-(0.71, 0.71)$; → $-(0.1, 0.1)$.
- **Adam**: t=1: $\tilde{\mathbf{v}} = \mathbf{g}, \tilde{\mathbf{a}} = \mathbf{g}^2$ ⇒ $-(0.1, 0.1)$; stays ≈ $-(0.1, 0.1)$ for constant $\mathbf{g}$.

Takeaway: GD/mom. move in proportion to the gradient (steep axis dominates); adaptive methods equalize the axes, so flat directions move as fast as steep ones.

CNN quick ref (on past exams)

Out = $\frac{W-F+2P}{S} + 1$. Conv params $= K(F \cdot F \cdot C_{in} + 1)$; pool: 0 params. Each output sees $F \cdot F \cdot C_{in}$ inputs. FC after: $(H \cdot W \cdot C) \times n_{out}$. Receptive field: $r = 1 + \sum_l (F_l - 1)\prod_{k<l} S_k$. Fewer params than FC (sparse connectivity, weight sharing) ⇒ less overfit with little data.